

소상공인 맞춤형 AI 광고 생성 서비스(Service_AD_System)

개인 프로젝트 기술 보고서 이태훈

1. 프로젝트 개요 (Overview)

-프로젝트명:

소상공인 맞춤형 AI 광고 텍스트 생성 플랫폼 (service_ad_system)

-개발 목적:소상공인들이 마케팅 콘텐츠(홍보 문구, 이벤트 텍스트 등)를 직접 제작하는 데 겪는 시간적·비용적 부담을 완화하기 위해 기획된 AI 기반 텍스트 자동 생성 서비스입니다.

오픈소스 LLM을 로컬 환경에 직접 구축·최적화하여 비용 효율적이면서도 독립적인 마케팅 솔루션을 구현하고자 했습니다.

-핵심 타겟: 저예산·저사양 환경에서 손쉽게 마케팅 카피를 자동화하고자 하는 소상공인 및 1인 사업자

2. 시스템 아키텍처 및 기술 스택 (Tech Stack)

기술 스택

-Backend: FastAPI, Python, Uvicorn (비동기 처리 기반 고성능 API 서버)

-Database & ORM: SQLite, SQLAlchemy (Async, 비동기 데이터베이스 연동)

-AI Model & Pipeline: HuggingFace Transformers (Qwen/Qwen2.5-0.5B-Instruct), -Pipeline API

-Frontend: Streamlit (반응형 웹 대시보드 UI)

-Observability: Loguru (구조화된 로깅), Prometheus (시스템 메트릭 수집)

디렉토리 구조 (src-layout 패턴 적용)

유지보수성과 확장성을 고려하여 표준화된 src-layout 아키텍처를 채택하였습니다.

Plaintext

```
service_ad_system/
├── alembic/                # 비동기 DB 마이그레이션 환경
├── src/
│   ├── service_ad_system/
│   │   ├── api/           # FastAPI 라우터 및 관측성 설정
│   │   ├── core/          # 환경 변수 및 공통 설정 관리
│   │   ├── frontend/      # Streamlit 대시보드 화면 (app.py)
│   │   └── services/       # Qwen AI 모델 추론 서비스
│   └── (model_service.py)
│       ├── models/         # SQLAlchemy DB 테이블 모델
│       └── main.py          # FastAPI 애플리케이션 진입점
├── pyproject.toml          # 패키지 및 의존성 관리 설정
└── README.md
```

3. 핵심 기능 (Key Features)

-AI 맞춤형 광고 텍스트 생성

업종, 매장 이름, 타겟 고객층, 이벤트 상세 내용을 입력받아 Qwen2.5-0.5B-Instruct 모델이 트렌디하고 감성적인 마케팅 문구를 자동 생성합니다.

-맞춤 해시태그 조합

매장 및 업종 키워드를 기반으로 SNS 마케팅에 즉시 활용할 수 있는 해시태그 조합을 실시간으로 도출합니다.

-저사양 온프레미스(On-Premise) 환경 최적화

무거운 멀티모달 및 외부 API 의존성을 제거하고, 로컬 CPU 및 4GB RAM 제

약 환경에서도 완벽히 구동되도록 경량화 파이프라인을 구축했습니다.

4. 상세 개발 과정 및 시행착오 (Troubleshooting & Trial and Error)

본 프로젝트를 진행하며 마주한 문제점과 이를 극복하기 위해 수행한 기술적 트러블슈팅 내역입니다.

[트러블슈팅 1] 로컬 컴퓨터의 저사양 환경에서의
OOM(Out-Of-Memory) 및 프로세스 강제 종료 현상

-문제 상황: 초기 기획 단계에서 고성능 모델(Qwen2.5-1.5B 또는 이미지 생성을 위한 Stable Diffusion)을 로컬 환경에 로드하려 시도했으나, 시스템 가용 램(4GB)의 한계치를 초과하여 에러 로그조차 남지 않은 채 Uvicorn 백엔드 프로세스가 곧바로 강제 종료(Shutting down)되는 현상이 발생했습니다.

-원인 분석:로컬 CPU 환경에서 1B 이상의 파라미터를 가진 모델이나 이미지 생성 파이프라인은 최소 8GB~12GB 이상의 메모리를 요구하므로, 제 로컬 환경에서는 메모리 경합으로 인해 운영체제가 프로세스를 구동되지 못하게 만들었습니다.

-해결 및 최적화:텍스트 모델을 허깅페이스의 초경량 체급인 Qwen/Qwen2.5-0.5B-Instruct (0.5B) 모델로 전면 교체하여 모델 메모리 점유율을 1GB 안팎으로 대폭 감축하였습니다.

이미지 생성 및 복잡한 SNS 외부 연동 기능을 과감히 배제하고, 소상공인 마케팅에 핵심적인 텍스트 카피 및 해시태그 생성에만 집중하도록 시스템을 변경하였습니다.

[트러블슈팅 2] 토큰 생성 지연 및 렉(Lag) 현상 개선

-문제 상황:모델 로드 성공 후에도 프론트엔드(Streamlit)에서 생성 요청을 보냈을 때 연산 부하로 인해 브라우저가 장시간 멈추거나 응답이 지연되는

현상 발생하였습니다.

-해결 및 최적화:model_service.py 내부의 파이프라인 호출 시 최대 생성 토큰 길이를 max_new_tokens=60으로 엄격히 제한하였습니다.

불필요한 확률적 샘플링 연산을 제거하기 위해 do_sample=False 옵션을 적용하여 CPU 연산 부하를 최소화하고 응답 속도를 극대화하였습니다.

[트러블슈팅 3] 코드 구현 중 발생한 문법 오류(SyntaxError) 해결

-문제 상황:백엔드 라우터 리팩토링 과정에서 파이썬 모듈 임포트 구문의 누락으로 인해 SyntaxError: invalid syntax 가 발생하며 앱 구동이 차단되었습니다.

-해결:src/service_ad_system/routers/generate.py 상단의 Pydantic 임포트 구문을 from pydantic import BaseModel, Field 형태로 정확히 교체하여 문법 오류를 원천 차단하였습니다.

5. 프로젝트 실행 및 세팅 가이드 (Getting Started Setup)

실제 개발 및 구동을 위해 적용된 세부 세팅 절차입니다.

(5-1) 저장소 클론 및 패키지 설치

```
Bash
git clone <repository-url>
cd service_ad_system
pip install -e .[dev]
```

(5-2)백엔드 모델 세팅

(src/service_ad_system/services/model_service.py)

```
Python
import asyncio
import torch
from transformers import pipeline
```

```

class ModelService:

    def __init__(self):
        print("Loading lightweight Qwen AI model (0.5B)...")
        self.device = 0 if torch.cuda.is_available() else -1
        model_id = "Qwen/Qwen2.5-0.5B-Instruct"

        try:
            self.generator = pipeline(
                "text-generation",
                model=model_id,
                device=self.device,
                torch_dtype=torch.float32,
            )
            print("Qwen 0.5B Model loaded successfully!")
        except Exception as e:
            print(f"Model load error: {e}")
            self.generator = None

    def predict(self, text: str) -> dict:
        if not self.generator:
            return {
                "input_text": text,
                "copy": f"시스템 대체 응답: {text}",
                "status": "error",
            }

        prompt = f"광고 카피를 짧고 매력적으로 작성해줘.\n\n요청: {text}\n\n카피:"
        outputs = self.generator(prompt, max_new_tokens=60,
do_sample=False)
        generated_text = outputs[0]["generated_text"]
        return {
            "input_text": text,
            "copy": generated_text.strip(),
            "status": "success" if generated_text else "error",
        }

    async def predict_async(self, text: str) -> dict:
        loop = asyncio.get_running_loop()
        return await loop.run_in_executor(None, self.predict, text)

model_service = ModelService()

```

Bash

```
python -m uvicorn service_ad_system.main:app --reload --port 8000
```

```
User@DESKTOP-NI3HFH3 MINGW64 ~/Desktop/개발 공부/파트4 팀프로젝트/service_ad_system
$ source "/c/Users/User/Desktop/개발 공부/파트4 팀프로젝트/service_ad_system/venv/Scripts/activate"
(venv)
User@DESKTOP-NI3HFH3 MINGW64 ~\Desktop\개발 공부\파트4 팀프로젝트\service_ad_system
$ python -m uvicorn service_ad_system.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\User\\Desktop\\개발 공부\\파트4 팀프로젝트\\service_ad_system']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reload process [15592] using WatchFiles
Loading Hugging Face model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████| 104/104 [00:01<00:00, 63.69it/s]
Model loaded successfully!
2026-10-01 03:54:17.008 | INFO | service_ad_system.api.observability:setup_observability:97 - Observability setup (Loguru + X-Request-ID + Prometheus) initialized successfully.
INFO: Started server process [2360]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

백엔드 통신 완료

(5-4) 프론트엔드 대시보드 실행 (새 터미널)

Bash

```
streamlit run src/service_ad_system/frontend/app.py --server.port 8501
```

6. 프로젝트 성과 및 시사점 (Conclusion)

하드웨어 제약 극복 능력 배양: 고사양 GPU가 강제되는 AI 애플리케이션 환경에서(저사양 RAM, CPU 전용) 온프레미스 제약을 분석하고 초경량 모델과 파라미터 최적화를 통해 서비스 안정성을 확보하는 실무적 문제 해결 능력을 길렀습니다.

아키텍처 설계 역량 강화: FastAPI와 Streamlit, 비동기 DB 연동 및 src-layout 구조를 적용하여 확장성 있고 유지보수가 용이한 파이썬 백엔드 구조를 설계했습니다.

7. 향후 업데이트 계획

인프라 확장: 단일 프로세스 메모리 한계를 극복하기 위한 Celery + Redis 기반 분산 백그라운드 큐 도입할 수도 있습니다.

마케팅 채널 연동: 네이버 스마트스토어, 인스타그램, 페이스북 API를 연동하여 생성된 광고 문구와 이미지를 원클릭으로 자동 업로드 기능 추가할 수도 있습니다.

사용자 인증: JWT 기반 로그인 및 소셜공인별 API 사용량 관리 기능을 구축할 수도 있습니다.